Mock Final Exam - Version V4

Total Points: 100

Mock Final Exam - Version 4

Total Points: 100

Multiple Choice (12 questions, 3 points each = 36 points)

Select the best answer for each question.

1. (3 points) To write a Java String to a binary file such that it can be read back easily using readUTF(), which method should you use?

A. writeBytes(String s)

B. writeChars(String s)

C. writeUTF(String s)

D. writeObject(String s)

2. (3 points) If a class implements Serializable but contains an instance variable of a type that is not serializable, what will happen when you try to serialize an object of that class (unless the variable is marked transient)?

A. The non-serializable variable is skipped automatically.

B. A ClassCastException is thrown.

C. A NotSerializableException is thrown.

D. The program compiles, but the non-serializable data is lost silently.

3. (3 points) Which Java I/O concept involves wrapping one stream around another to add functionality (e.g., buffering, data type handling, object handling)?

A. Stream pipelining (or chaining/wrapping)

B. Stream redirection

C. Stream cloning

D. Stream multiplexing

4. (3 points) The seek(long pos) method in RandomAccessFile positions the file pointer relative to:

A. The current position of the pointer.

B. The end of the file.

C. The beginning of the file (offset 0).

D. The last position written to.

5. (3 points) What is a common application where recursion is often a more natural fit than iteration?

A. Calculating the sum of an array.

B. Processing data structures defined recursively, like trees or fractals.

C. Reading data sequentially from a file.

D. Simple counter-controlled loops.

6. (3 points) If a recursive method calls itself with the exact same arguments repeatedly without approaching a base case, what error is likely?

A. NullPointerException

B. StackOverflowError

C. ArrayIndexOutOfBoundsException

D. IllegalArgumentException

7. (3 points) The recursive solution for the Tower of Hanoi problem demonstrates that:

A. Recursion is only suitable for mathematical calculations.

B. Some problems have elegant recursive solutions that are complex to implement iteratively.

C. All recursive solutions require a helper method.

D. Recursion always leads to faster execution times.

8. (3 points) Which JavaFX layout pane is useful for placing nodes at specific pixel coordinates (x, y)?

A. FlowPane

B. TilePane

C. AnchorPane

D. Pane (or subclasses like Region)

9. (3 points) What method is typically called on a JavaFX Stage object to make it visible?

A. display()

B. setVisible(true)

C. show()

D. activate()

10. (3 points) An interface in Java can contain:

A. Only abstract methods.

B. Abstract methods, default methods, static methods, and constants (public static final variables).

C. Only static methods and constants.

D. Instance variables and constructors.

11. (3 points) When a subclass overrides a method from its superclass, the overridden method in the subclass must have:

A. A different name but the same parameters.

B. The same name, same parameters, and same return type (or a subtype for covariant returns).

C. The same name but different parameters.

D. A less restrictive access modifier (e.g., superclass is protected, subclass is public).

12. (3 points) What is the purpose of the finally block in a try-catch-finally statement?

A. To catch any exceptions not caught by the catch blocks.

B. To specify the exception types to be caught.

C. To contain code that must execute whether an exception occurs or not (often used for cleanup).

D. To re-throw an exception after logging it.

Short Answer / Code Reading

1. (4 questions, variable points = 29 points)

2. (6 points) Explain the concept of a "marker interface" like Serializable or Cloneable. What is characteristic about their definition, and what is their purpose?

3. (7 points) Consider the following recursive method:

```
public static int sumDigits(int n) {
if (n < 10) { // Base case: single digit
return n;
} else { // Recursive step
return (n % 10) + sumDigits(n / 10);
}
}
```

Trace the execution of sumDigits(123). Show the calls and the return values leading to the final sum.

4. (8 points) Describe how you would use ObjectInputStream to read an ArrayList of Serializable objects that was previously written to a file using ObjectOutputStream. What specific method call reads the entire list, and what casting is required?

5. (8 points) Compare and contrast recursion and iteration as methods for solving problems. Mention at least one advantage and one disadvantage for each approach.

Coding / Program Design

1. (4 questions, variable points = 35 points)

2. (10 points) Write a recursive Java method isPalindrome(String s, int low, int high) that returns true if the portion of the string s between indices low and high (inclusive) is a palindrome, and false otherwise. The initial call for checking the whole string would be isPalindrome(myString, 0, myString.length() - 1). You must use recursion. Define the base case(s) and recursive step clearly within your code using comments.

```
public static boolean isPalindrome(String s, int low, int high) {
// Base Case(s): ??
// Recursive Step: ??
}
```

3. (12 points) Write a Java code snippet that demonstrates writing and then reading back different primitive data types using DataOutputStream and DataInputStream.

- Use try-with-resources to handle a FileOutputStream wrapped in a BufferedOutputStream and DataOutputStream for writing to "primitives.dat".

- Write a boolean, an int, and a double value to the file.

- Then, use try-with-resources to handle a FileInputStream wrapped in a BufferedInputStream and DataInputStream for reading.

- Read the boolean, int, and double back in the correct order and print them to the console.

- Include basic exception handling.

4. (13 points) Design an abstract class Shape with an abstract method getArea() that returns a double. Create two concrete subclasses, Circle (with a radius field) and Rectangle (with width and height fields). Implement constructors for Circle and Rectangle, and provide the concrete implementations for the getArea() method in both subclasses using the standard area formulas (AreaCircle = pi * radius^2, AreaRectangle = width * height). Use Math.PI for pi.

```
abstract class Shape {
// Abstract method?
}

class Circle extends Shape {
private double radius;
// Constructor?
// Implementation of getArea()?
}

class Rectangle extends Shape {
private double width;
private double height;
// Constructor?
// Implementation of getArea()?
```

```
}
```